

课程笔记

GLM-5.1：为长程智能体任务而生

Codex 多智能体协作整理

2026 年 5 月 25 日



视频作者/频道: 智猩猩 / 李子玄 (智谱 Z.ai)

发布日期: 2026-05-21

视频时长: 00:22:03

视频链接: <https://www.bilibili.com/video/BV1QWL46vEx2/>

目录

1 从单点生成到长程任务：AI Coding 的范式拐点	2
1.1 演讲背景与三段式路线图	2
1.2 2025 年的单点能力竞赛	3
1.3 SWE-bench 式评测的饱和与污染风险	4
1.4 长任务暴露的三类失败	5
1.5 本章小结	6
2 长程任务的核心定义：目标保持、迭代优化与失败恢复	6
2.1 长程任务不能等同于长文本、长对话或长步骤	7
2.2 从一分钟 Demo 到八小时系统的质量跃迁	8
2.3 长程智能体的能力三角：全局理解、逐步推导、验证迭代	8
2.4 向量数据库案例：600 次迭代中的搜索、索引、量化与失败标记	9
2.5 本章小结	10
3 GLM-5.1 的训练路线：从架构选择到 OPD	11
3.1 GLM-5.1 的开源位置与竞品坐标	11
3.2 架构选择：DeepSeek V3.2 架构与效率导向	11
3.3 Mid-training：让模型先见过高质量思考	13
3.4 统一模型与自蒸馏：把专家能力合回主线模型	14
3.5 OPD：用分布匹配减轻灾难性遗忘	14
3.6 本章小结	16
4 把长程能力落到场景与评测：任务分类、权重重排与 METR 启发	17
4.1 长程不能停在抽象标签：先把场景拆清楚	17
4.2 用户日志与真实意图：安装配置为何占比高	17
4.3 评测权重重排：从日志分布到能力分布	19
4.4 METR 启发：用人类完成时间刻画任务视界	20
4.5 本章小结	23
5 产品化判断与个人准备：场景意识、科学评测与 PMF 前埋伏	23
5.1 研究员也要像产品经理：懂用户、懂场景、懂边界	24
5.2 Vision Coding 例子：截图为什么会成为代码输入	24
5.3 评测是 AI 时代的管理方法	25
5.4 PMF 前埋伏：技术拐点来临前先站好位置	26
5.5 讲者结尾与行业速度	28
5.6 本章小结	29
6 总结与延伸	30
6.1 核心定义的压缩	30
6.2 从模型到系统	30
6.3 从评测到产品	30

1 从单点生成到长程任务：AI Coding 的范式拐点

本章先回答一个朴素问题：为什么一场介绍 GLM-5.1 的演讲，要从 AI Coding 的评测变化讲起？讲者在 00:00:12–00:01:14 给出本次分享的路线：先看 AI Coding 领域正在出现的范式变化与 Long Horizon 议题，再拆解 GLM-5.1 的训练方式，最后谈大模型从业者如何面对快速变化的行业。这个开场很关键，因为它把 GLM-5.1 放进了一个更大的竞争背景里：模型能力的比较，正在从“某一张图、某一个网页、某一个 issue 能不能做漂亮”，转向“一个智能体能否在拖长的任务链中持续推进目标”。



图 1: 报告开场页把 GLM-5.1 放在 Long Horizon 任务能力的框架下，为后续讨论 AI Coding 范式变化建立主题锚点。¹

1.1 演讲背景与三段式路线图

从时间线上看，讲者在 00:00:19–00:00:45 先回顾 GLM-5、GLM-4.x 的前期分享，再把本场主题落到 GLM-5.1 与长程智能体。讲者还提到，Kimi K2.6 以及其他同期头部模型也在用 Long Horizon 这个概念。这里不需要急着定义 Long Horizon，本章只要抓住一个时代信号：当多个模型发布都把同一个词推到前台时，行业竞争的坐标轴已经开始移动。

三段式路线图的教学意义

讲者安排了一个从现象到机制再到行动的结构。第一段看 AI Coding 的竞争信号为什么变了；第二段讲 GLM-5.1 如何通过训练路线响应这种变化；第三段把技术判断转成从业者准备。换句话说，GLM-5.1 在这场演讲里既是模型名称，也是讲者用来解释“下一代智能体任务”的案例。

在 00:00:49–00:01:14，讲者明确列出三部分：AI Coding 范式变化、GLM-5.1 的训练拆解、大模型工作者的准备。这个结构给后文设定了边界：本章负责制造问题意识；第 2 章再正式展开长程任务的定义；第 3 章进入训练技术；第 4、5 章再讨论评测、场景和产品化。

¹ 视频画面时间区间：00:00:12–00:00:49。

1.2 2025 年的单点能力竞赛

讲者在 00:01:18–00:02:13 回顾了 2025 年 AI Coding 圈最容易被看见的竞争信号。第一类是网页生成：模型拿到一个 prompt，生成一个完整页面，评委或用户马上能从布局、配色、交互细节判断“好不好看”。第二类是 3D、SVG 或可视化 demo：例如 Minecraft 风格场景、茶壶骑自行车这类带有空间结构和组合约束的例子。它们的共同点很直接：输出物可以截图、可以排名、可以在社交媒体上传播。

讲者还给出两个具体锚点：Gemini 3 Pro / Gemini 3.1 Pro 发布时，网页生成质量提升曾被广泛感知；GLM-5.1 在某次 3D/SVG battle 中取得第一，同时仍有部分模型在这类生成上能力不足。这些例子说明，2025 年的旧竞争点很依赖可截图、可排名、可传播的单点产物。

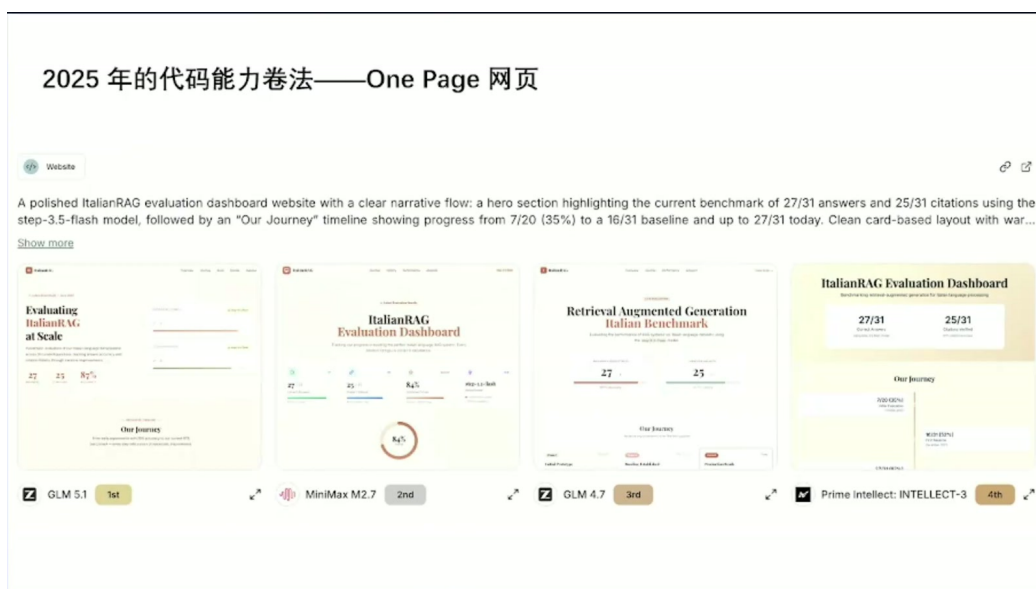


图 2: One Page 网页生成展示了 2025 年代码能力竞争的可见形态：模型被要求一次性产出完整、精致、可评测的网页结果。²

这种竞赛可以粗略写成一个单点映射：

$$y = f_M(p)$$

- p ：用户给模型的一次性提示词，例如“生成一个科技感主页”或“画一个 3D 场景”。
- M ：参与生成的模型。
- f_M ：模型在当前提示词下完成生成的函数式近似。
- y ：一次输出的页面、SVG、3D 场景或局部代码补丁。

这个公式不追求严格刻画真实系统，只用于说明“单点生成”的评测直觉：输入相对集中，输出相对完整，成功与失败主要在一个局部对象上观察。网页好不好看、SVG 有没有画出来、某个 feature 能不能跑，通常很快就能判断。

²视频画面时间区间：00:01:25–00:01:47。

单点生成质量的核心含义

单点生成质量关注一次输出的局部完成度。它适合评估模型的审美、代码片段能力、格式控制和局部约束遵循能力，也容易被可视化案例放大成舆论信号。它的问题在于：真实工程任务常常需要多轮查证、跨文件协调、失败恢复和长期目标保持，局部漂亮不自动推出过程可靠。

1.3 SWE-bench 式评测的饱和与污染风险

00:02:13–00:03:13 是本章的第一个转折点。讲者把视线从网页、SVG 这类可见 demo 转向 SWE-bench 代表的代码评测。SWE-bench 的价值在于，它来自真实 GitHub issue，要求模型理解仓库、修改代码、运行环境、通过隐藏测试。相比“一次生成一个页面”，它更接近真实软件维护：一个修复可能牵涉多文件、依赖关系、测试失败和上下文理解。

研究笔记也提醒了这一点：SWE-bench 是真实 issue-resolution benchmark，它对仓库理解、多文件协调、环境交互和隐藏测试都有要求。因此，它曾经是 AI Coding 能力跃迁的重要标尺。讲者的判断更尖锐：到 2025 年某个阶段，SWE-bench Verified 这类评测开始失去足够的区分度。他给出的两个原因分别是数据污染和时代代表性下降。

字幕里还有一个更具体的拐点叙事：SWE-bench Verified 一度被卷到 open4.5 约 80.9 分，后续 Opus 4.6 一类新模型在该口径下反而低于 open4.5；讲者据此引出 2025 年 2 月 OpenAI 宣布 SWE-bench Verified 已失去可评估价值。这里的模型名来自 AI 字幕，最终应以幻灯片或公开资料校正；正文需要保留的，是“高分拥挤加新模型反常低分”这条推理线。

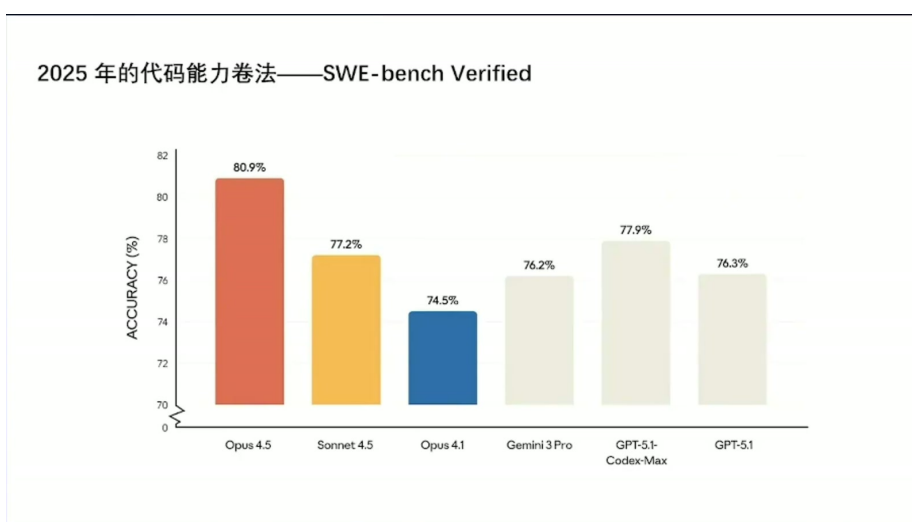


图 3: SWE-bench Verified 分数页呈现出旧评测口径的拥挤状态：若分数差距难以继续拉开，评测本身就会变成范式迁移的信号。³

数据污染与评测饱和的区别

数据污染指 benchmark 题目、答案、近似解法或相关讨论进入训练数据，使模型可能“见过题”。评测饱和指大量强模型都能拿到高分，分数差异难以继续代表真实能力差异。二者经常同时出现，但机制不同：污染伤害公平性，饱和伤害区分度。

³视频画面时间区间：00:02:13–00:02:36。

讲者在 00:02:46–00:02:57 解释数据污染时，说法很朴素：很多答案已经被模型记住，再进入公开评测就会让分数失真。00:02:57–00:03:07 则进一步指出任务本身的时代参考性下降：如果大部分题已经太简单，做出来只能说明模型跟上了旧标准；做不出来也可能是题目质量、环境或标注本身出了问题。这个判断需要写得克制：它是讲者对当时评测状态的判断，研究笔记只支持 SWE-bench 的任务性质，并未替讲者验证每个分数与发布声明。

1.4 长任务暴露的三类失败

从 00:03:13 开始，讲者进入本章最重要的压力测试：当任务被拉长，原来的 SOTA 模型会暴露什么问题。这里的“长”先不要理解成正式的 Long Horizon 定义。它更像工程现场里的拖拽实验：把一个原本可以一轮完成的任务，拉成多个相互依赖的步骤，观察模型会在哪些地方失控。

第一类失败是循环与漂移。讲者在 00:03:24–00:03:40 说，即使模型宣称上下文很长，例如能处理百万级上下文，任务一旦拉长仍然可能陷入循环、跑偏、失意、崩盘。这里要区分两个概念：上下文长度是一次调用能容纳多少文本和状态；任务变长带来的是目标保持、状态连续和错误恢复压力。一个模型能读很长的输入，不代表它能在长链路中持续做正确的下一步。

第二类失败是局部修复破坏相邻目标。00:03:43–00:03:58 的例子是：让模型搭建一个 feature，它可能完成这个 feature；但与该 feature 相关的优化、边界条件、相邻模块协调，未必能处理得深入。软件工程里这很常见：修一个按钮，影响表单状态；改一个 API，打破缓存；补一个测试，掩盖真实业务失败。局部补丁像给墙上裂缝刷漆，短期看干净，结构问题还在背后延伸。

第三类失败是高水平工程能力不足。讲者在 00:03:59–00:04:21 把“做网页、做爬虫、debug”与“写 CUDA kernel、写数据库、写操作系统”放在同一个梯度上比较。前者常常是低到中等复杂度的局部任务，后者需要系统设计、性能模型、边界条件、测试策略和长期调参。以 CUDA kernel 为例，表面上是写一段 GPU 代码，底层牵涉线程块划分、访存合并、共享内存、寄存器压力和吞吐权衡；数据库和操作系统则进一步牵涉一致性、并发、恢复、调度和接口稳定性。

SOTA 模型也可能出现的问题

- 任务一拉长，就“跑偏、失忆、崩盘”
- 只能“点单式”干活，做不了闭环工程
- 低水平工作的堆叠形成不了高水平工作

图 4: SOTA 模型在任务拉长后暴露的三类问题：目标漂移、单点式执行、低水平任务堆叠，这些现象共同指向更深的长程能力缺口。⁴

⁴视频画面时间区间：00:03:16–00:04:21。

本章的范式拐点

旧范式奖励“局部对象的完成质量”：页面是否好看、SVG 是否成形、issue 是否修掉。新压力来自“过程是否可靠”：模型能否在多轮操作中保持目标、记住未完成义务、处理失败反馈，并把局部修复放回整体工程里检查。

容易误读的一点

长上下文是必要工具之一，但它只提供更大的信息容器。长任务真正考验的是容器里的信息能否被组织、筛选、更新和验证。把上下文窗口加大，像把桌面加宽；项目能不能做完，还取决于任务分解、检查清单、测试反馈和返工能力。

1.5 本章小结

这一章建立了 GLM-5.1 演讲的起点：2025 年的 AI Coding 竞争先被网页生成、3D/SVG demo、Minecraft 或茶壶骑自行车这类可视化案例推到台前，随后 SWE-bench 式真实 issue 修复评测成为更严肃的代码能力标尺。随着可视化 demo 的舆论效应变强、SWE-bench 类评测出现饱和与污染风险，单点生成质量已经不足以支撑下一阶段的模型比较。

讲者在 00:03:13–00:04:21 给出的三类失败，为后文 Long Horizon 做了铺垫：任务拉长后，模型可能循环漂移；完成一个 feature 后，邻近优化与整体一致性仍然薄弱；面对 CUDA kernel、数据库、操作系统这类高水平工程任务时，局部代码能力无法自动扩展成系统级可靠性。

带到下一章的问题

本章只提出压力：短任务和局部生成正在失去足够的解释力。下一章需要回答更核心的问题：怎样的任务才配称为长程任务，智能体在长链路里究竟要保持什么、恢复什么、验证什么。

2 长程任务的核心定义：目标保持、迭代优化与失败恢复

上一章把问题推到了一个拐点：网页、SVG、局部功能、SWE-bench 式修复都能显示模型的单点生成能力，但任务一拉长，系统会进入循环、跑偏、局部修好又破坏旁边模块。演讲在 00:04:22–00:04:39 把这个拐点具体化：若干头部模型在前端开发、SWE-bench 这类旧竞争点上的提升已经不明显，真正亮眼的突破出现在 Long Horizon，也就是长程任务上。

本章负责给出全书使用的核心定义。这里的“长程”指向智能体系统在一个延迟兑现的目标下，持续保存目标、状态、约束和验证纪律，并在工具调用、失败尝试、外部反馈之间反复调整的能力。研究笔记里 OpenAI 对 agentic AI system 的定义强调“在有限直接监督下追求复杂目标”，这给长程任务提供了系统边界：能力不只在模型参数里，也在工具、指令、运行时、权限、沙箱、记忆和评估回路里。⁵ Anthropic 对 workflow 与 agent 的区分也有帮助：workflow 依赖预设路径，agent 会根据中间观察动态选择工具和过程。⁶ 长程任务恰好经常无法提前写死步数。

⁵OpenAI, <https://openai.com/index/practices-for-governing-agentic-ai-systems/>; OpenAI, <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>.

⁶Anthropic, <https://www.anthropic.com/engineering/building-effective-agents>; Anthropic, <https://www.anthropic.com/engineering/managed-agents>.

长程任务的工作定义

长程任务 (Long-horizon task) 可以这样理解：一个完整智能体系统 A 围绕目标 G ，在一串相互依赖的行动与观察中维护状态 S_t ，把失败反馈转化为下一轮更好的行动，直到产物通过外部验证。它关注的是过程可靠性和累计改进，而非文本长度、对话轮数或运行时长本身。

2.1 长程任务不能等同于长文本、长对话或长步骤

00:04:45–00:05:28 这段是本章最容易被误读的地方。讲者连续排除了三类“看起来很长”的任务：长文本、长对话、长步骤。一个模型跑 16 小时、20 小时，或者从 1 数到 100 万，确实会占用很多时间；把社交账号上一万个粉丝逐个抓取下来再生成网页，也确实包含很多步骤。可这些任务的核心结构很浅：每一步与下一步的关系近似重复，失败反馈很少改变总体策略，最终产物也缺少“第 1 分钟版本到第 8 小时版本”的质变。

这里要把三个术语分开。上下文长度 C 是一次模型调用能看到多少 token 或状态。长程压力来自任务本身需要跨越的依赖关系、未知状态和验证回合。自治视界则是部署后的系统能在多远距离内继续推进，直到需要用户接管。第 4 章会用 METR 的公式正式量化“任务视界”。长上下文可以给长程任务提供材料，像更大的桌面；它不会自动带来目标保持、状态整理和失败恢复，正如把一本厚手册放在桌上并不会自动修好一个数据库。

长程任务，本质上是「深度」，而不是单纯的「长度」

- 长程 $\neq$ 长文本、长对话、长步骤
- 真正的长程任务，是“深度任务”——保持目标与逻辑不失真
- 具体体现——全局理解、逐步推导、反复验证、迭代优化

图 5: 长程任务的关键在于“深度”：系统需要持续保持目标与逻辑，并通过全局理解、逐步推导、反复验证和迭代优化累积成果。⁷

长度替代依赖深度

常见误区是用“长度”替代“依赖深度”。低水平重复的任务可以很长，但它对智能体的压力主要是耐心和批处理；长程任务的压力来自错误会累积、目标会漂移、状态会膨胀、验证会推迟。两者的工程难度不在同一层。

从计算角度看，低水平重复更像循环：

$$\text{for } i = 1, \dots, n, \quad y_i = f(x_i)$$

⁷ 视频画面时间区间：00:04:45–00:06:35。

其中 i 表示第 i 个独立条目, x_i 是输入, y_i 是输出, f 是几乎不变的处理函数。这样的任务即使 n 很大, 核心挑战也主要是规模。长程任务更像状态递推:

$$a_t = \pi(G, S_t), \quad S_{t+1} = U(S_t, a_t, o_t)$$

其中 G 是目标或成功标准, S_t 是第 t 步后的系统状态, a_t 是智能体在第 t 步采取的行动, o_t 是行动后的观察结果, π 是根据目标和状态选择行动的策略, U 是把旧状态、行动和观察合并为新状态的更新过程。这个式子刻意很小, 因为本章只需要说明一点: 下一步会依赖前面已经发生过什么, 行动序列因此具有真正的状态性。

2.2 从一分钟 Demo 到八小时系统的质量跃迁

00:05:28–00:05:55, 讲者给了一个很好的直觉标尺: 真正的长程任务, 第一分钟做出的成果和第八个小时做出的成果应该有惊人的差别。第一分钟可能只是一个很小的 demo; 第八小时经过持续优化、建设和修正, 应该形成更系统化的东西。这个差别体现在结构更对、性能更好、失败更少、验证更硬, 而非单纯写得更多。

这个判断很朴素, 也很严格。一个人类工程师在第 1 分钟也许只能搭起目录、写出朴素实现、跑出第一个错误; 第 8 小时应该已经理解性能瓶颈, 试过几条路线, 丢掉无效方案, 保留可解释的中间结论, 并让测试、日志或指标显示改进。若智能体在 8 小时后只是堆出更多文件、更多话术、更多未经验证的 patch, 它仍然没有越过长程任务的门槛。

这一段继续沿用前面的长程定义图, 不另插独立截图。讲者真正想让读者记住的, 是质量跃迁这把尺子: 第八小时的系统应当比第一分钟的 demo 多出结构、验证和性能改进。

智能体系统的组成

OpenAI 的 agents 构建指南把 agent 拆成模型、工具和指令三部分; Anthropic 的 Managed Agents 又把长程运行拆成 session、harness、sandbox。可以把它们类比成远征队: 模型像决策者, 工具像仪器和机械臂, 指令像任务书, session 像航海日志, harness 像调度员, sandbox 像可控实验场。远征能否抵达终点, 取决于整套系统是否把目标和证据一路带下去。

这里也解释了为什么“目标保持”是长程智能体的第一能力。目标 G 不只是用户开头说的一句话, 它包含成功标准、禁止事项、质量阈值、未完成承诺、环境限制。例如“写一个向量数据库”这个目标, 最低层看是代码生成; 长程层面看, 它包含检索正确性、吞吐量、索引结构、内存占用、构建时间、测试覆盖, 以及哪些优化已经试过。目标保持失败时, 系统会把局部通过当作全局完成, 把临时修补当作结构改进, 把“这一步没报错”误认为“整个任务解决了”。

2.3 长程智能体的能力三角: 全局理解、逐步推导、验证迭代

00:05:55–00:06:35, 讲者把长程能力压成三件事。第一, 系统要有全局理解: 写 CUDA kernel、写数据库、写操作系统这类任务, 必须先理解优化目标、成功标准和胜负条件。第二, 系统要能逐步推导: 任务常常不会一轮完成, 可能需要 100 轮、300 轮, 甚至更多工具调用。第三, 系统要能不断验证和迭代: 它要从现有规律与外部反馈中判断路线是否有效, 避免把答案背下来。

这三角可以对应到工程执行的三种失误。全局理解弱, 系统会把 benchmark 数字、单个报错或用户某句话当成全部目标。逐步推导弱, 系统会在第 20 步以后遗忘第 3 步做过的取舍, 导致修改互相打架。验证迭代弱, 系统会把失败当作噪声跳过, 或在同一条无效路线上重复消耗步骤。

能力三角的状态底座

长程智能体的核心是每一步都能更新同一个任务状态：保留已经验证的事实，标记失败路线，维护尚未解决的约束，并把下一次行动对准原始目标。“思考很多步”只是表面现象。

状态连续性是这三角的底座。状态 S_t 超出纯聊天历史，涵盖文件变化、命令输出、测试失败、检索证据、设计决策、用户限制、审批结果、临时假设和待复核事项。研究笔记里 OpenAI 对 stateful runtime 和 Codex-Max compaction 的描述都指向同一件工程事实：上下文窗口会耗尽，历史需要压缩，但未解决承诺和关键证据不能蒸发。⁸ 所谓 compaction，直觉上像给实验日志做索引和摘要；删掉冗余闲聊可以，丢掉“哪些方案失败过、为什么失败、当前目标还欠什么”就会直接破坏长程能力。

目标漂移的安静风险

长程任务里最危险的失败常常很安静：系统没有崩溃，甚至每一步都有合理解释，但目标已经悄悄漂移。比如原目标是“提升 QPS 且保持召回率”，后续循环若只盯吞吐量，最终可能得到一个很快但检索质量坍塌的系统。

2.4 向量数据库案例：600 次迭代中的搜索、索引、量化与失败标记

00:06:35–00:07:43 是本章的正例。讲者展示 GLM-5.1 技术报告里的一个向量数据库任务：横轴显示大约 600 次迭代，纵轴围绕 QPS，也就是 queries per second，每秒查询数。任务目标避开了漂亮页面生成和 isolated function 这类单点产物，聚焦于持续提升一个向量数据库的检索吞吐。

向量数据库可以先用直觉理解：普通数据库擅长精确查找，例如“用户 ID 等于 123”；向量数据库擅长相似度查找，例如“找出和这段文本语义最接近的文档”。每条数据被表示成一个向量，两个向量越接近，语义或特征越相似。最朴素的方法是逐个比较所有向量，这像在图书馆里每本书都翻一遍；数据量一大，QPS 会很低。

讲者描述的路线有明显的迭代层次。最开始，模型没有找到很好的解法，只用比较原始的方法做向量匹配。中间阶段，它开始学会使用倒排索引。倒排索引的直觉是把“文档里有什么词或特征”反过来组织成“某个词或特征出现在哪些文档里”，先缩小候选集合，再做精细比较。后面又引入量化技术，也就是用更低成本的表示近似原始向量，牺牲一点精度来换速度、内存和缓存友好性。到更后面，讲者提到还会使用剪枝或更高级的方法继续提升 QPS；这里不应把具体算法说死，因为字幕把“剪枝”识别得不稳定，图像 agent 还需要对对应幻灯片做视觉确认。

检索系统里的四种工程杠杆

QPS 是吞吐量指标，表示每秒能处理多少次查询。倒排索引是检索系统里常见的“先找候选，再精排”的结构。量化是压缩表示的一类方法，例如把高精度浮点向量换成更低精度或离散码，从而降低计算与内存压力。剪枝则是提前去掉不值得继续计算的候选或结构。它们像四种工程杠杆：少看一点、看得更快、表示更轻、把没希望的路线提前停掉。

图中还有一个更重要的细节：若干叉号代表模型提出过某些方法，最后发现不 work。00:07:17–00:07:23，讲者专门提醒这些叉号的意义。失败标记具有长程任务“路况图”的价值。一个系统如果

⁸OpenAI, <https://openai.com/index/the-next-evolution-of-the-agents-sdk/>; OpenAI, <https://openai.com/index/gpt-5-1-codex-max/>.

⁹视频画面时间区间：00:06:35–00:07:43。

长程任务，本质上是「深度」，而不是单纯的「长度」

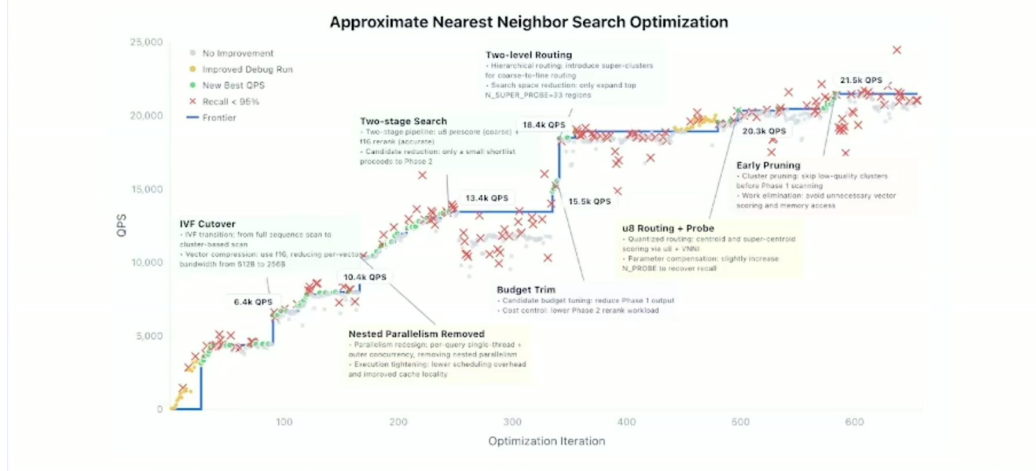


图 6: 向量数据库优化曲线把长程能力具体化: 600 次迭代中, QPS 前沿阶梯式上升, 红色叉号记录了最终不 work 的失败尝试。⁹

能记录“某方案为何失败”，下一轮就可以避开同样的坑，或把失败拆成更小的可诊断问题；如果只把失败日志丢进上下文深处，后续步骤很容易重新发明同一个无效方案。

失败标记与 QPS 曲线都集中在图 6 中；本段用同一张图读取“失败路线被记录下来”的信息。

向量数据库案例的长程证据

向量数据库案例给出长程任务的可观察形态：从朴素搜索到索引结构，再到量化与候选削减，QPS 曲线在多轮尝试中上升；叉号记录失败尝试，说明系统在学习“哪些路走不通”。这比一次性写出答案更接近真实工程。

这个例子也把“失败恢复”讲清楚了。恢复超过简单重试；简单重试只是在同一输入上再次按下按钮。失败恢复包含诊断、归因、状态更新和新策略选择：失败发生在哪里，是否由正确性、性能、内存、接口、测试环境或假设错误引起；失败影响了哪些已完成工作；下一轮应保留什么、撤销什么、替换什么。对智能体来说，恢复能力越强，失败越像训练信号；恢复能力越弱，失败就会变成上下文污染。

从 00:07:23–00:07:43 的总结看，讲者强调的 Long Horizon 能力接近人类工程师的工作方式：一开始并不知道最终要做哪种高级操作；系统需要不断从过去失败经验里总结教训，尝试新方法，在成功与失败之间继续优化，直到达到结果。说得直白一点，长程智能体的价值体现在犯错之后仍能保持目标、整理状态、校正路线，并把下一轮做得更好；“永远不犯错”并非现实工程里的可靠性标准。

2.5 本章小结

本章给出了后续章节共用的定义：长程任务是完整智能体系统围绕目标 G ，在状态 S_t 、行动 a_t 、观察 o_t 的循环中持续积累改进的任务。它不能用文本长度、对话长度、运行时长或步骤数量来替代衡量。

00:04:45–00:05:28 的反例说明，计数到 100 万、抓取一万个粉丝这类任务可以很长，却缺少依

赖深化和策略更新。00:05:28–00:05:55 的一分钟 demo 与第八小时系统，给出了质量跃迁的标尺。00:05:55–00:06:35 的能力三角告诉读者，长程智能体需要全局理解、逐步推导、验证迭代。00:06:35–00:07:43 的向量数据库案例进一步落地：600 次迭代、QPS 提升、倒排索引、量化、剪枝候选和失败叉号，构成了“从反馈中累计改进”的证据链。

下一章会把这个定义接到 GLM-5.1 的训练路线：如果长程能力的关键是目标保持、状态连续和失败恢复，那么训练管线就必须回答一个尖锐问题：怎样在增强长程能力的同时，保住已有通用能力，避免后续训练覆盖前面学到的东西。

3 GLM-5.1 的训练路线：从架构选择到 OPD

前一章已经把长程任务的核心讲清楚：真正难的地方在于目标保持、状态连续、失败恢复和多轮验证。本章进入讲者的第二个部分：GLM-5.1 这类模型怎样训练，才能在长程智能体任务里表现得更稳。讲者给出的路线可以压缩成一句话：先用架构保证训练效率，再用中期训练让模型见过高质量代码与推理，接着把专家能力蒸回统一模型，最后用 OPD 缓解多阶段强化学习带来的能力覆盖。

3.1 GLM-5.1 的开源位置与竞品坐标

在 00:07:49–00:08:28 这一段，讲者先用一张排名图说明 GLM-5.1 的位置。他的口述提到 Kimi K2.6 发布后排名格局发生变化；图中可见条目需要以幻灯片标签为准。这里需要谨慎理解：这类榜单往往依赖测试集、提示方式、工具环境和发布时间，任何“第一”“领先”“落后”的结论都应绑定到讲者当时展示的幻灯片和评测口径。

讲者口述里还有一个时间锚点：GLM-5.1 曾在开源 SOTA 位置待了 14 天，直到 Kimi K2.6 发布后相对排序被改写。这个数字适合写成讲者在该评测口径下的陈述，避免扩展成跨榜单、跨日期的长期事实。

更值得学习的是讲者怎样解读这个位置。他没有把 GLM-5.1 描述成绝对压倒式领先，重点放在它处于头部模型和后一梯队之间：距离最强两家还有差距，同时也和后续梯队拉开了一定身位。这个判断对训练路线有启发：如果目标是长程智能体，模型训练就不能只追求某个单点榜单的峰值，还要保证代码、推理、工具调用、通用对话等能力互相支撑。

排名图的正确读法

这一页适合当作“讲者口径下的能力坐标”，不宜当作跨平台、跨日期、跨评测体系的稳定事实。模型排名像天气预报：给出了某个时间、某个地区、某套仪器下的读数；离开这些条件，数字的含义会迅速变薄。

3.2 架构选择：DeepSeek V3.2 架构与效率导向

讲者随后回答一个常见问题：GLM-5.1 到底靠什么训出来。他先提到团队采用的是 DeepSeek V3.2 架构，并补充说团队也做过架构研究，只是基于实验结果选择了这条路线。这里的关键点很朴素：架构重要，但它主要影响训练效率、训练速度和数据利用率；模型最终强不强，还要看数据、阶段设计、评测反馈和后训练策略。

¹⁰源视频时间区间：00:07:49–00:08:28。

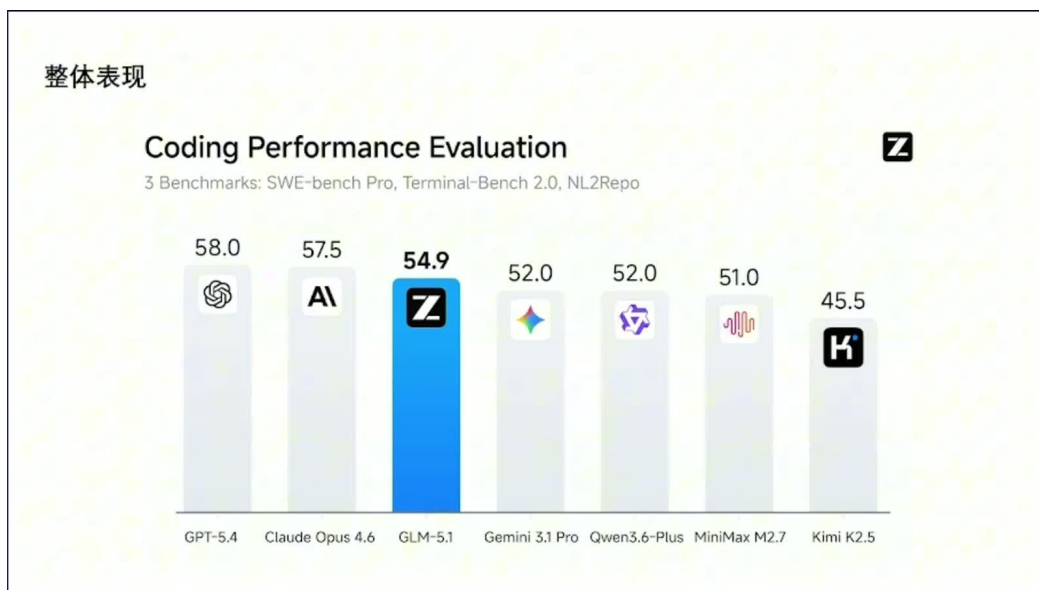


图 7: 讲者展示的 Coding Performance Evaluation 排名图: GLM-5.1 位于 GPT-5.4、Claude Opus 4.6 之后, 并高于图中其余模型; 这张图适合用来限定“当时、该评测口径下”的相对坐标。¹⁰

可以用“修路”和“开车”来类比。架构像路网：路修得好，货车跑得更快，拥堵更少，运输成本更低。数据和训练策略像车上的货物、司机的路线规划与调度制度。路网很差时，再好的司机也会被卡住；路网足够好之后，胜负常常来自货物质量、调度节奏和反馈闭环。讲者在这里把架构放回工程系统的位置，避免把它神化成唯一胜负手。

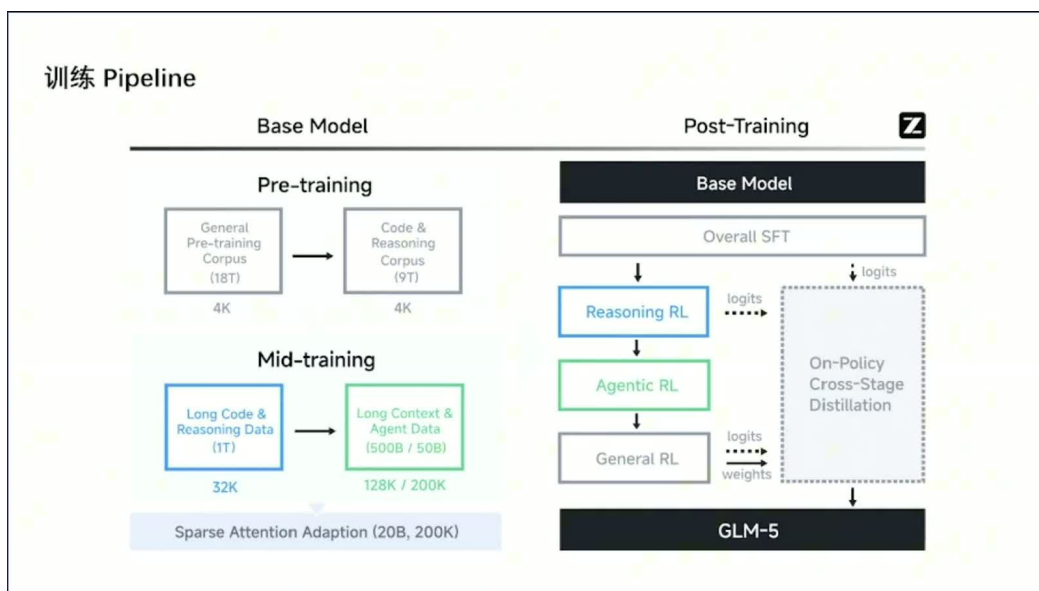


图 8: GLM 系列训练 Pipeline 总览：左侧呈现 base model 阶段的预训练、中期训练与 Sparse Attention Adaption，右侧呈现后训练中的 SFT、RL 阶段和跨阶段蒸馏。¹¹

¹¹源视频时间区间：00:08:51–00:09:30。

架构、参数与训练效率

在大模型语境里，架构指模型内部的信息流组织方式，例如注意力、专家混合、层间连接、归一化和路由设计。它决定同样的算力和数据能转化成多少有效学习。若把模型参数看成可调节旋钮，架构就是这些旋钮被安装在什么机器上；旋钮数量很重要，机器布局同样决定能否稳定、高效地调到好状态。

3.3 Mid-training：让模型先见过高质量思考

讲者称 GLM-5.1 的第一个“杀手锏”出现在 mid-training 环节。按他的划分，训练链路大致是预训练、中期训练、后训练三段。预训练给模型打底，让它从海量文本和代码里学语言、知识和基本模式；中期训练在底座模型已经形成后，引入更高密度、更强结构的数据；后训练则通过监督微调 and 强化学习，让模型学会按任务要求输出、调用工具、遵守约束并完成评测。

Mid-training 的中文可以译作“中期训练”。它的位置夹在 pre-training 和 post-training 之间，作用像让学生在刷题前先读高质量范文、看优秀工程项目、学习严谨解题过程。讲者举了几类数据：高质量思维链数据、大量跨代码库代码，以及整理后的高质量代码合集。这些数据让模型先领略什么叫高质量代码，什么叫高质量推理，什么叫在长程任务中逐步推进。

图中的 Long Context & Agent Data 是把长程任务里的状态连续性、工具链行为和跨轮依赖转成训练信号的一类数据来源。它把第 2 章讲过的“目标、状态、观察、验证”从概念层推进到数据配方层。

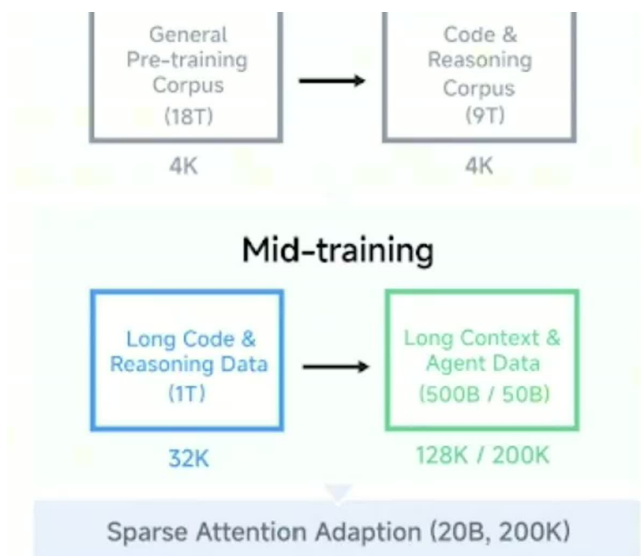


图 9: 中期训练区域的局部裁切：Long Code & Reasoning Data 与 Long Context & Agent Data 被放在预训练和后训练之间，底部还标出 Sparse Attention Adaption。¹²

Mid-training 的核心作用

中期训练的目标是提高模型的“审美”和“过程感”。代码审美包括结构清晰、边界明确、可测试、可维护；推理过程感包括知道先定位问题、再提出假设、再验证、再修正。长程智能体尤其依赖这种过程感，因为它要在很多轮动作之后仍然记得什么才算真正完成。

¹²源视频时间区间：00:09:30–00:10:25。

这里还要区分两类看似相近的数据。高质量代码数据让模型学会“什么是好工程”，例如跨仓库依赖、抽象边界、测试约束、性能瓶颈。Chain-of-thought 数据让模型学会“好答案如何被推出来”，例如拆解目标、排除错误路径、记录中间结论。前者像看优秀程序员写出的项目，后者像站在优秀程序员旁边听他解释每一步为什么这样做。二者叠在一起，才更接近长程任务需要的训练信号。

CoT 数据的误读

思维链数据的价值不在于让模型输出更长的解释。更核心的价值是提供可学习的中间结构：问题分解、证据排序、局部验证、失败回滚、结论压缩。长解释若缺少这些结构，只会增加噪声；短推理若保留这些结构，仍然可能很强。

3.4 统一模型与自蒸馏：把专家能力合回主线模型

讲者接着回顾了从专门模型到统一模型的选择。早期团队可能会训练 coder 模型、thinking model、通用对话模型等多个专家模型。每个专家模型像一位偏科高手：有人擅长写代码，有人擅长复杂推理，有人擅长自然对话。到了 GLM-4.5 之后，团队选择把这些能力合入一个主线模型，让用户面对的是统一入口，而训练过程中产生的专家能力通过自蒸馏回流。

自蒸馏可以理解为“模型家族内部的师徒传承”。普通蒸馏常见于大模型教小模型：教师模型给出输出分布或示例，学生模型学习它的行为。自蒸馏强调教师和学生来自同一训练体系，可能是不同阶段、不同专家方向、不同数据配方下得到的模型。它的工程目标很清楚：让主线模型吸收专家模型的长处，减少产品层面的模型切换成本。

统一模型为什么有产品价值

多模型路线可以让每个专家追求局部最优，但用户任务经常混合出现：一个真实的智能体任务可能先要读需求，再改代码，再查资料，再写解释，再修复测试。统一模型像一名全科工程师，未必在每个动作上都拥有单科冠军的尖峰，却能在连续任务里减少切换、路由和能力断裂。

从训练角度看，统一模型也带来新难题。不同专家的偏好可能冲突：代码模型偏向精确和可执行，通用对话模型偏向解释和礼貌，推理模型偏向长步骤展开。把这些能力合并时，需要让模型知道何时简洁、何时展开、何时调用工具、何时停止。自蒸馏提供了一种合并方式，但合并完成后还要面对强化学习阶段的能力覆盖问题。

3.5 OPD：用分布匹配减轻灾难性遗忘

讲者在 00:11:16–00:12:37 提到 GLM-5 相对早期版本的重要改进：加入 on-policy distillation，也就是 OPD。理解 OPD 之前，先要理解多阶段强化学习为什么危险。讲者给出的顺序大致是：先有 SFT 版本模型，然后进入不同阶段的 RL，例如 reasoning RL、agentic RL、general RL。每一阶段都会更新模型参数，让模型在某类奖励下变强；同时，参数更新也可能改掉前一阶段学到的习惯。

这就是灾难性遗忘。它像同一个运动员连续换教练：第一个教练纠正发力，第二个教练强调速度，第三个教练强调耐力。每次训练都有收益，但后一次训练可能把前一次训练建立的动作模式冲淡。模型参数 θ 也是这样：阶段 k 得到的参数 θ_k 记录了某类能力，阶段 $k+1$ 的优化会继续移动参数位置，旧能力可能被新目标覆盖。

¹³源视频时间区间：00:10:25–00:11:16。

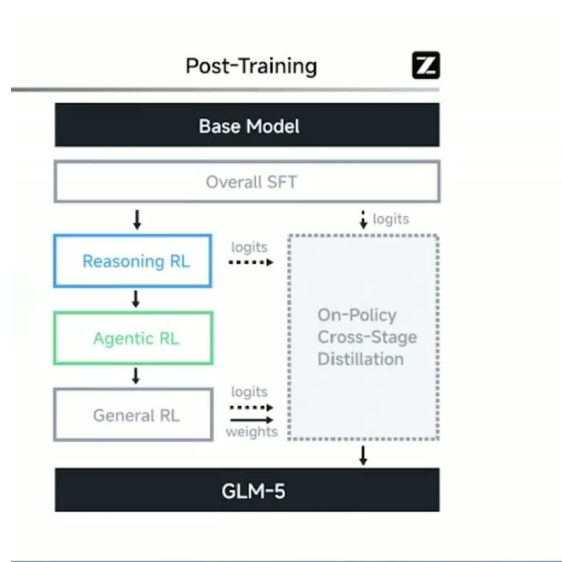


图 10: 后训练局部图：Base Model 经过 Overall SFT 后进入 Reasoning RL、Agentic RL、General RL，多阶段模型信号通过 logits 和 weights 汇入最终 GLM-5。¹³

灾难性遗忘的工程后果

灾难性遗忘的可怕之处在于它常常发生在“指标进步”的同时。模型在新 RL 阶段对应的榜单上变好，旧场景却悄悄退化。对长程智能体来说，这种退化尤其麻烦：一次任务会同时用到代码、推理、对话、工具调用和错误恢复，任何一块短板都可能让长链路中途断掉。

OPD 的做法是把多个阶段的模型都拿回来，做一次面向分布的蒸馏。讲者用“logits 匹配”描述这个过程。Logits 是模型在 softmax 之前给每个候选 token 的原始分数；softmax 会把这些分数变成概率分布。匹配 logits 或其对应概率分布，含义是让最终模型不只学某一个标准答案，还要学教师模型对各种候选输出的相对偏好。

下面的公式是教学化写法，用来表达 OPD 的目标：

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{k=0}^K w_k \text{KL} \left(\text{softmax} \left(\frac{z_{\theta_k}(x)}{\tau} \right) \parallel \text{softmax} \left(\frac{z_{\theta}(x)}{\tau} \right) \right)$$

符号说明如下：

- θ ：最终要训练得到的模型参数。
- θ_k ：第 k 个训练阶段留下的模型参数，例如 SFT 后、reasoning RL 后、agentic RL 后、general RL 后的模型。
- x ：用于蒸馏的输入样本，可以是问题、代码任务、工具调用上下文或长程任务片段。
- $z_{\theta_k}(x)$ ：阶段模型对输入 x 给出的 logits，也就是 softmax 之前的原始打分。
- τ ：温度参数，用来调节概率分布的平滑程度；温度越高，分布越平，模型会看到更多“次优选”的信息。
- w_k ：第 k 个阶段模型在蒸馏损失里的权重，表示最终模型应从该阶段保留多少信号。

- $KL(\cdot\|\cdot)$: KL 散度，用来衡量两个概率分布的差异；差异越小，学生模型越接近教师模型的偏好。

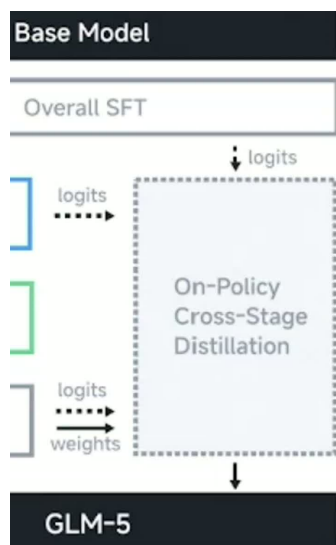


图 11: OPD 区域裁切: 虚线框标出 On-Policy Cross-Stage Distillation, 周围的 logits 与 weights 箭头强调它在做跨阶段分布对齐。¹⁴

OPD 的一句话理解

OPD 把多个训练阶段的模型当作一组“阶段教师”，让最终模型同时贴近它们的输出分布。这样做的目的，是在继续提升长程、推理和智能体能力时，尽量保住 SFT、通用能力、代码能力和前序 RL 阶段已经学到的东西。

为什么要匹配分布，而非只比较最终答案？因为分布包含更细的知识。假设有一道代码修复题，教师模型把“先运行测试”打 0.35，把“先读错误栈”打 0.30，把“直接改代码”打 0.05。最终答案可能只出现一种行动，但这些概率关系告诉学生：哪些路径接近合理，哪些路径危险。长程任务中，许多关键能力都藏在这种偏好排序里。

这也解释了 OPD 与长程智能体之间的关系。长程任务需要模型在很多步里保持稳定的策略偏好：遇到失败要验证，遇到不确定要查证，改代码后要跑测试，工具失败后要恢复现场。若后续 RL 只强化某一类短期奖励，模型可能变得激进、冗长、保守或过度迎合。OPD 像训练结束前的一次能力对账，把多个阶段形成的分布重新对齐到最终模型里。

3.6 本章小结

本章把讲者关于 GLM-5.1 的训练路线拆成五层。第一层是位置坐标：讲者用排名图说明 GLM-5.1 在当时开源模型中的相对位置，但这些比较应保留评测口径。第二层是架构选择：DeepSeek V3.2 架构主要服务于效率、速度和数据利用，架构之外还需要训练链路的系统设计。第三层是 mid-training：在预训练和后训练之间，让模型先见过高质量代码、跨仓库工程和质量思维链。第四层是统一模型与自蒸馏：把不同专家模型的能力合回主线模型，减少真实任务中的能力断裂。第五层是 OPD：用多阶段模型的 logits 分布匹配来减轻灾难性遗忘。

¹⁴源视频时间区间：00:11:16–00:12:37。

这条路线的底层逻辑很直接：长程智能体能力靠单点尖峰很难支撑。模型既要学会高质量思考，又要保留通用能力；既要在推理和 agentic RL 中继续变强，又要防止后一个阶段覆盖前一个阶段。GLM-5.1 在讲者叙述中更像一个能力保留工程：它追求的重点并非单项刷分到极致，核心在于多种能力之间建立可持续的训练闭环。

4 把长程能力落到场景与评测：任务分类、权重重排与 METR 启发

前两章已经把长程任务的压力讲清楚：它考验目标保持、状态连续、失败恢复和外部验证；第三章则把这种能力要求接到 GLM-5.1 的训练路线。本章进入更务实的一层：一个团队若只说“要做强 Long Horizon”，这个目标仍然太虚。训练数据怎么造，benchmark 怎么设，哪些用户需求应当被放大，哪些高频需求应当降权，这些选择才会真正改变模型。

讲者在 00:12:38–00:16:40 给出的方法很直接：先把长程任务拆成场景，再用系统日志观察用户意图分布，然后把原始频率转成训练和评测权重，最后借鉴 METR 这类研究，用“合格人类需要多久完成”刻画任务视界。这里的重点在于把评测变成训练工程的方向盘，榜单只是外显结果。

4.1 长程不能停在抽象标签：先把场景拆清楚

00:12:38–00:13:12，讲者谈到 GLM-5 到 GLM-5.1 之间，Long Horizon 变成一个很热的方向。讲者的判断可以概括成一句话：模型在长程任务上表现强，原因通常落在若干具体场景的理解深度上。抽象标签本身不会带来能力，能训练、能评测、能复盘的是场景。

场景分类的第一刀，是把使用行为分成“非任务”和“任务”。非任务主要指闲聊、陪伴式对话、开放式交流；它可以很长，但很多时候缺少明确的交付物和验收标准。任务类使用则有更清楚的目标、步骤和结果，例如查资料、写报告、部署服务、处理表格、生成演示文稿、修改代码、配置工具链。长程能力真正容易显形的地方，通常在任务类使用里，因为那里存在状态、依赖、失败和验收。

第二刀，是把任务类继续拆成技术型、信息型和个人效率型。技术型包括代码、运维、环境配置、性能优化、数据库和系统工程；信息型包括搜索、资料收集、事实核验、跨来源整合；个人效率型包括办公文档、表格、日程、摘要、写作、知识整理。这个三分法很粗，却足够有工程价值：它把一个大词切成可采样、可标注、可诊断的任务族。

场景分类的核心作用

场景分类把“长程能力”从宣传词变成工程对象。技术型任务强调工具链、代码环境和可运行验证；信息型任务强调检索、证据和冲突消解；个人效率型任务强调用户上下文、格式约束和最终交付物。分类越清楚，训练数据与评测题目越容易对准真实能力缺口。

这里还要补一个容易被忽略的点：场景分类并不等同于产品菜单。菜单面向用户展示，分类面向训练和评测。一个用户看到的是“帮我写周报”；训练团队看到的应当是资料抽取、语气约束、结构化改写、历史上下文继承、格式交付和质量检查。前者是入口，后者是能力分解。

4.2 用户日志与真实意图：安装配置为何占比高

00:13:53–00:14:11，讲者展示了一个基于系统日志的用户意图排序，并特别指出“安装配置”会排在很高的位置。原因并不神秘：很多人第一次使用长程智能体，先让它配置自己，例如安装 skills、

¹⁵ 源视频时间区间：00:12:38–00:13:35。

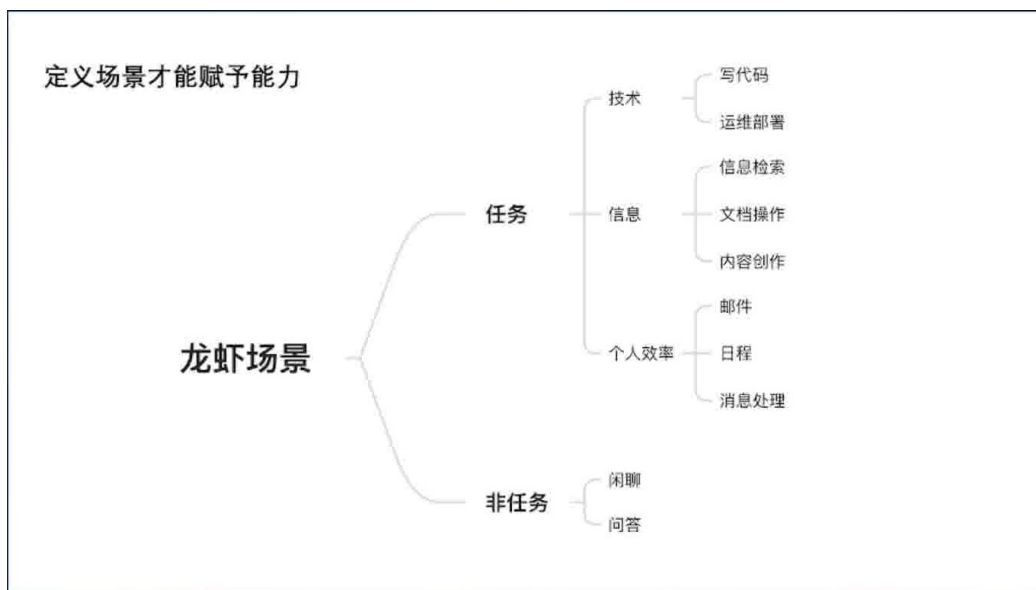


图 12: 长程场景被拆成任务与非任务两条主线；任务侧继续分为技术、信息与个人效率等类别，给后续数据构造和评测权重提供可操作的坐标。¹⁵

设置环境、连接工具、修复运行条件。换句话说，长程智能体的第一批真实任务，常常是“让智能体把自己变成可用的智能体”。

这个现象很有启发。原始日志频率反映的是入口摩擦、产品成熟度和用户习惯的混合物。安装配置高频，可能说明用户确实需要自动化环境准备，也可能说明当前产品还需要太多手工配置。若直接按频率训练，模型会过度优化“自我安装”和“自我修复”这类浅层启动任务，导致评测看起来贴近日志，能力却没有覆盖更高价值的长程工作。



图 13: 系统日志中的用户意图分布显示，安装配置、代码开发、信息查询与搜集等入口任务排在前列，提示原始频率会受到产品使用入口影响。¹⁶

¹⁶源视频时间区间：00:13:53–00:14:27。

日志频率不能直接等于能力权重

用户日志像城市路口的车流量，能告诉人哪里拥堵，却不能直接告诉人城市最需要哪条高速公路。安装配置排第一，可能是刚需，也可能是产品摩擦造成的短期峰值。训练与评测若只照搬日志分布，就会把“最常出现的问题”误当成“最值得代表能力的问题”。

更严谨的做法，是把用户意图区分成至少三种信号。第一是频率信号：用户到底问了多少次。第二是价值信号：这个意图解决后，对用户是否产生真实生产力提升。第三是能力信号：这个意图是否能区分模型在状态管理、工具调用、证据整合、失败恢复上的水平。安装一个 skill 高频，但区分度可能有限；完成一份跨来源研究报告频率较低，却能更强地暴露检索、归纳、引用、格式和验收能力。

4.3 评测权重重排：从日志分布到能力分布

00:14:27–00:15:27 是本章的工程核心。讲者说，团队在做 bench 时会把权重做重排序：安装配置的权重降下来，信息搜索与收集升上去，办公类任务升得更高，写代码甚至可以移出这个特定长程评测。这个选择的理由很清楚：代码能力已经在 SWE-bench 式 coding 场景里被长期钻研，若在长程综合评测里继续放入大量代码题，容易重复计分，掩盖信息型和办公型任务的真实短板。

可以把这个过程写成一个简单的分布变换。设场景集合为 $\mathcal{S}$ ，原始日志给出用户意图分布 $q(s)$ ，评测设计者再根据代表性、价值、区分度和重复计分风险设定校正系数 α_s ，得到 benchmark 权重：

$$w(s) = \frac{\alpha_s q(s)}{\sum_{s' \in \mathcal{S}} \alpha_{s'} q(s')}$$

- $\mathcal{S}$ ：场景集合，例如安装配置、信息搜索、办公任务、代码任务等。
- s ：集合 $\mathcal{S}$ 中的某一个具体场景。
- $q(s)$ ：系统日志观察到的原始用户意图比例。
- α_s ：评测设计者为场景 s 设置的校正系数，用来表达代表性、业务价值、能力区分度和是否重复计分。
- $w(s)$ ：最终进入 benchmark 或训练诊断的场景权重。
- 分母 $\sum_{s' \in \mathcal{S}} \alpha_{s'} q(s')$ ：归一化项，保证所有场景权重加起来为 1。

这个公式只是一种教学化表达，现实中权重还会受标注成本、任务可验证性、安全边界和上线节奏影响。它的价值在于提醒读者：benchmark 天然带有设计立场，绝非中性的“题目堆”。题目分布一变，模型优化方向就会变；权重一变，团队看到的短板也会变。

¹⁷源视频时间区间：00:14:27–00:15:27。



图 14: OpenClaw 场景示例把左侧日志意图映射到右侧 Bench 题目分布：办公与日常事务、信息查询和搜集、安装配置、开发运维、数据分析与总结、安全等类别获得不同题量与比例。¹⁷

WebArena 为什么适合作为旁证

WebArena 这类 web-agent benchmark 强调真实网站中的多步交互、状态跟踪和任务完成，评价对象超出单轮问答。^a 它提醒评测设计者：长程评测需要把环境状态、页面操作、错误恢复和最终验收纳入题目结构。代码之外，信息检索、后台操作、表单填写、账号设置和办公流也能暴露智能体的长程能力。它对本章的价值在于提供 coding-only 之外的多步交互样本，帮助长程评测摆脱单一代码权重。

^aWebArena paper, <https://arxiv.org/abs/2307.13854>.

讲者接着说，基于这样的 bench，团队可以做三件事。第一，更好地出题，避免题目只覆盖最容易出现、最容易刷分的入口任务。第二，评估各家模型在不同场景中的强弱，减少单一总分带来的遮蔽。第三，用评测结果倒逼训练：哪个场景弱，就构造对应数据集；哪类失败高发，就设计针对性优化方案。这里的“倒逼”指让评测像体检报告一样指出训练系统的病灶，刷榜只能制造短期数字。

评测倒逼训练

评测权重决定了训练团队看见什么问题。若权重偏向安装配置，模型会更擅长处理启动摩擦；若权重提高信息搜索和办公任务，训练会被迫处理证据质量、跨文档一致性、格式交付和用户目标保持。评测是训练工程的反馈回路，也是产品判断的显微镜。

4.4 METR 启发：用人类完成时间刻画任务视界

00:15:27–00:16:40, 讲者把视角从团队内部 bench 扩展到 METR。字幕把 METR 识别成“meter”，这里按研究笔记修正为 METR。讲者描述的关键做法，是构造一批任务，从几秒钟就能回答的选择题或问答题，到需要约 8–10 小时的人类工作才能完成的 CUDA kernel 等复杂任务，然后观察不同模型或智能体系统能稳定完成多长任务。

讲者还把这个尺度放进模型代际比较里：从 GPT-2 到后续头部模型，能覆盖的任务人类工时逐

步变长。这个比较的重点，是让读者看到任务视界随模型和智能体系统能力演进而扩展。

METR/Kwa et al. 2025 的研究笔记给出了一个更正式的说法：50%-task-completion time horizon，即在某个任务分布上，智能体达到约 50% 成功率时，对应任务所需的合格人类完成时间。¹⁸ 这一定义很锋利，因为它把“长”从模型运行时间、token 数量、对话轮数中拿出来，放到任务本身的难度和人类工作量上。

$$H_{p,D}(A) = \inf\{t : \Pr_{x \sim D_t}[\text{agent } A \text{ completes } x] \geq p\}$$

- A ：完整智能体系统，包括模型、工具、运行框架、沙箱、记忆或状态、权限控制和评估回路。
- D ：任务分布，例如软件任务、网页操作任务、信息收集任务或办公交付任务的总体。
- D_t ：分布 D 中合格人类完成时间约为 t 的任务子集。
- t ：合格人类完成该类任务所需的时间，而非模型或程序实际运行的墙钟时间。
- p ：成功概率阈值。METR 的标题指标常用 $p = 0.5$ ，即约一半任务能完成。
- $H_{p,D}(A)$ ：系统 A 在分布 D 和阈值 p 下的任务视界。
- $\Pr_{x \sim D_t}[\text{agent } A \text{ completes } x]$ ：从 D_t 抽取任务 x 时，系统 A 完成任务的概率。

严格地说，这个公式是一种 METR 风格的教学写法。真实研究会处理任务分桶、采样误差、成功判定、时间估计和曲线拟合等细节。对本章来说，最重要的是尺度选择：一个三秒钟选择题、一个十分钟检索任务、一个两小时 repo 修复、一个八小时 CUDA kernel 优化，不应被压成同一类“任务完成率”。时间尺度一拉开，长程能力才有可比较的横轴。

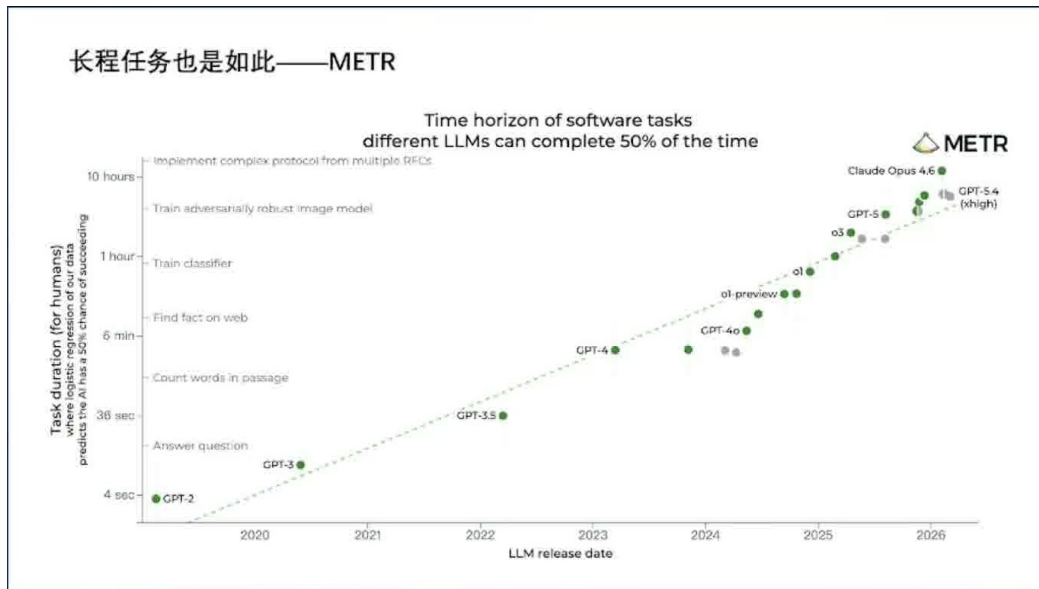


图 15: METR 曲线用合格人类完成任务所需时间作为纵轴，展示不同模型在 50% 完成率附近可覆盖的软件任务视界。¹⁹

¹⁸METR / Kwa et al. 2025, <https://arxiv.org/abs/2503.14499>.

¹⁹源视频时间区间：00:15:27–00:16:00。

²⁰源视频时间区间：00:16:04–00:16:23。

长程任务也是如此——METR

Family	Length	Description
find_shell_script	3 seconds	Multiple choice: “Which file is a shell script?” Choices: “run.sh”, “run.txt”, “run.py”, “run.md”
wikipedia_research	1 minute	Research simple factual information from Wikipedia and provide accurate answers to straightforward questions.
oxdna_simple	9 minutes	Detect and fix a bug in the input files for a molecular dynamics simulation using the oxDNA package.
munge_data	56 minutes	Write a Python script to transform JSON data from one format to another by inferring the conversion rules from provided example files.
cuda_backtesting	8 hours	Speed up a Python backtesting tool for trade executions by implementing custom CUDA kernels while preserving all functionality, aiming for a 30x performance improvement.

图 16: METR 任务样例表把 3 秒选择题、1 分钟维基查询、9 分钟缺陷修复、56 分钟数据转换和 8 小时 CUDA kernel 优化放在同一时间尺度上，帮助读者理解任务视界的跨度。²⁰

三种“长”必须分开

上下文长度 C 衡量一次模型调用能容纳多少文本或状态；任务视界 $H_{p,D}(A)$ 衡量系统能完成多长人类工作量的任务；自治视界衡量部署后的系统在没有需要人类反馈或干预前能走多远。长上下文可以帮助任务视界，却不能替代任务视界；高任务视界也不自动推出产品中的高自治，因为真实部署还受风险、审批、用户信任和权限边界约束。

这也解释了 OpenAI 与 Anthropic 研究笔记为何反复强调“完整智能体系统”。OpenAI 的 agentic AI systems 讨论把复杂目标、有限监督、工具和治理边界放在一起；Anthropic 对 workflow 与 agent 的区分提醒我们，长程任务常常无法预先写死路径；Anthropic 关于 autonomy measurement 的讨论还指出，能力测量和部署测量存在差距。²¹ 一个系统在实验 benchmark 上能完成八小时人类任务，仍然需要产品层面的监督、回滚、权限和责任设计，才能在真实用户手里承担更长自治链路。

为什么用“合格人类时间”

用人类完成时间做横轴，有点像用马拉松距离衡量跑步能力，而不看跑者鞋带系了多久。模型运行十小时却只机械重复，任务视界可能很短；模型运行五分钟却解决了一个人类需要两小时定位的复杂 bug，任务视界反而更长。这个尺度把注意力放回任务难度和成功概率。

讲者在 00:16:23–00:16:40 的总结很关键：建立这样的评测体系，团队才知道什么算长程任务，以及最终要解决的任务复杂到什么程度。这句话把本章闭合起来了。场景分类回答“测什么”；日志分布回答“用户在做什么”；权重重排回答“哪些任务最能代表能力”；METR 式任务视界回答“任务到底长到什么级别”。四者合在一起，长程能力才从概念进入训练流水线。

²¹OpenAI, <https://openai.com/index/practices-for-governing-agentic-ai-systems/>; Anthropic, <https://www.anthropic.com/engineering/building-effective-agents>; Anthropic, <https://www.anthropic.com/research/measuring-agent-autonomy>.

4.5 本章小结

本章把长程能力从抽象口号落到了评测工程。讲者先把长程使用分成非任务与任务，再把任务拆成技术型、信息型和个人效率型。这个分类让团队能够围绕真实交付物、状态依赖和验收标准组织数据，避免只围绕一个热词训练。

随后，用户日志提供了第一层事实：安装配置可能在原始意图分布中占比很高，因为很多用户进入智能体后的第一件事就是配置智能体自身。但原始频率不能直接当作能力权重。安装 skills 这类任务有价值，代表性和区分度却有限；信息搜索、资料收集、办公交付等任务更能暴露证据整合、状态保持和结果验收能力。

因此，评测权重需要从日志分布重排为能力分布。安装配置可以降权，信息搜索和办公任务可以升权，已被 coding benchmark 充分覆盖的写代码任务可以在特定长程综合评测里移出，以避免重复计分。这样的 benchmark 既能比较模型，也能反向指导数据构造和场景优化。

最后，METR 的 50%-task-completion time horizon 给出了一把更硬的尺子：用合格人类完成时间 t 和成功概率阈值 p 衡量完整智能体系统 A 的任务视界 $H_{p,D}(A)$ 。这把尺子要求读者分清三件事：上下文长度是信息容量，任务视界是可完成任务的人类时间尺度，自治视界是部署时离开人工干预后能推进多远。长程智能体的训练，最终要同时尊重场景、分布、权重和尺度。

5 产品化判断与个人准备：场景意识、科学评测与 PMF 前埋伏

前四章已经把技术主线铺开：AI Coding 的竞争焦点从单点生成走向长程过程可靠性；长程任务要求目标保持、状态连续与失败恢复；GLM-5.1 的训练路线围绕能力保留与长程增强展开；评测体系则把抽象能力拆成可度量的场景分布。本章接住演讲最后 00:16:40–00:22:03 的部分，把这些技术判断转成产品化和个人准备问题。

讲者的落点很直接：大模型时代的模型研究员、产品负责人、工程师和创业者，都要更早地理解用户场景，更硬地设计评测，更提前地站到技术拐点可能到来的位置。这里的“提前”没有玄学意味，它指的是在 PMF 尚未显性出现时，已经有场景假设、数据闭环、技术储备和分发承接能力。



图 17: “人人都是产品经理”这一页把研究员的产品化任务压成两件事：明确能力边界，培养场景意识；后面的 vision coding 例子正是这种场景还原能力的具体展开。²²

²²源视频时间区间：00:16:44–00:18:27。

5.1 研究员也要像产品经理：懂用户、懂场景、懂边界

00:16:44–00:17:29，讲者借用了互联网时代流行过的一句话：“人人都是产品经理”。他真正想强调的对象更窄，也更狠：在 AI 时代，研究员越来越像产品经理。原因很简单，模型能力已经很难只靠抽象指标解释。一个研究员如果要训练长程智能体能力，就要亲自理解长程智能体在用户手里到底处理什么任务；如果要训练代码智能体能力，就要知道用户在 Claude Code、Cursor、Z.ai 或同类工具中怎样打开项目、怎样贴错误、怎样跑测试、怎样中断和恢复任务。

场景意识（scenario literacy）指把用户行为还原成训练和评测对象的能力：用户在什么入口触发模型，给了哪些上下文，允许模型调用什么工具，失败后会怎样表达不满，成功时会留下什么可验证产物。缺少这种还原能力，训练数据就容易变成漂亮但失真的样本；评测题也容易只覆盖模型擅长表演的部分。

场景意识的工作定义

场景意识指研究员或产品团队能够把一句能力口号还原成具体使用链路：用户是谁，任务入口在哪里，输入材料是什么，模型可用哪些工具，成功标准如何验证，失败会怎样显现，边界条件由谁承担。它连接产品直觉、数据构造、评测设计和训练优化。

讲者在 00:17:21–00:17:29 连续提到“能力边界”“局限性”和“场景意识”。这三个词应放在一起看。能力边界说明模型在哪些条件下可靠，局限性说明模型在哪些条件下会失效，场景意识说明哪些条件在真实用户那里高频出现。产品化的第一步应把模型放进最能放大优势、最能控制风险、最容易被用户验证的链路里。

把能力口号当产品答案，是很贵的误判

“会长程任务”“会 coding”“会视觉理解”都只是能力标签。产品团队还要回答更细的问题：哪个用户会为它打开工具，哪一步最痛，模型失败时损失多大，是否能用日志和评测捕捉改进。没有这些约束，能力标签很快会变成发布会语言，无法稳定转化为留存、付费或真实生产力。

这里可以借用 Anthropic 关于 agent autonomy 的区分来补一层：能力测量和部署测量属于两个问题。一个系统在实验评测中能完成较长任务，说明它在受控分布上具备能力；用户在真实产品里愿意放手让它运行多久，还取决于界面设计、可中断性、权限控制、失败成本、审计日志和人类监督方式。²³ 因此，研究员像产品经理，含义是研究员要理解能力进入真实链路后的摩擦、责任和验证方式。

5.2 Vision Coding 例子：截图为什么会成为代码输入

00:17:33–00:18:27，讲者用 K2.5 提出的 vision coding 做例子。这个例子的价值在于，它把“为什么代码模型需要视觉能力”从抽象能力问题拉回到一个极具体的开发者动作：写 Python 时，缩进表达代码结构；复制粘贴到聊天框后，缩进、空格、换行和格式经常被破坏；用户为了保留屏幕上的真实结构，会直接截图发给模型。

这类需求很核心。Python 的缩进语义让视觉布局本身成为程序信息的一部分。对人类来说，一眼看屏幕就能区分函数体、循环体、异常处理块和注释区域；对纯文本模型来说，复制粘贴后的格式

²³ Anthropic, <https://www.anthropic.com/research/measuring-agent-autonomy>.

噪声可能让代码结构变形。截图输入给模型的价值，在于保留编辑器里的空间结构、报错位置、选中区域、终端输出和 IDE 视觉状态。

这一段没有独立的代码截图页，正文直接保留场景解释。

Vision Coding 的产品含义

Vision coding 可以理解为“把屏幕状态也纳入代码任务上下文”。它覆盖图片识别、IDE 布局、缩进、终端错误、浏览器预览、组件渲染差异、截图里的表格或日志。用户发截图，常常是在告诉模型：纯文本通道已经丢掉了关键信息。

讲者的推理链是：真实用户行为暴露了输入通道缺口；产品团队捕捉到这个缺口；训练团队把这类场景纳入数据集；模型再针对性优化。这个链路比“多模态很重要”更有解释力。多模态是技术能力，vision coding 是能力嵌入代码工作流后的形态。前者回答模型能看见什么，后者回答用户为什么要让模型看见。

从训练角度看，这类场景还会改变数据结构。普通代码样本可能是“问题文本 → 代码答案”；vision coding 样本更像“截图、局部文本、文件状态、用户意图 → 诊断或补丁”。如果评测仍然只测纯文本补全，就很难奖励模型理解屏幕状态的能力。讲者在这里其实给了一个小型产品方法论：先找用户为什么绕过常规输入，再把绕行行为变成一类正式场景。

5.3 评测是 AI 时代的管理方法

00:18:27–00:19:27，讲者把“评测”上升到管理方法。他说最重要的事情是知道自己如何、竞品如何、每个场景表现如何；在专家和 AI 都能回答问题的时代，关键能力变成“会问问题”。这句话听起来像学习建议，实际指向模型组织里的管理系统：团队需要把模糊体感转成可比较、可复现、可回归的指标。

评测的管理作用可以用一个简单公式表示。产品或模型版本 v 在整体场景分布上的价值，可以近似写成：

$$S(v) = \sum_{i=1}^n w_i \cdot m_i(v)$$

- $S(v)$ ：版本 v 的综合评测分数或决策参考值。
- i ：第 i 类场景，例如信息搜集、办公自动化、代码修复、安装配置、视觉代码诊断。
- w_i ：该场景在当前产品战略中的权重。
- $m_i(v)$ ：版本 v 在该场景上的可观测表现，例如成功率、人工偏好、耗时、成本或错误率。
- n ：纳入评测的场景数量。

这个公式并不神秘，它只是把“感觉 A 比 B 好”拆成两个问题：好在哪些场景，权重为什么这样分配。讲者在 00:19:03–00:19:13 批评纯体感决策：如果只说 A 大于 B，就无法回答到底学习 A 的什么，也无法在 A、B、C 三个版本之间做严谨选择。模型训练和产品迭代都需要这种分解，否则团队会陷入“这版感觉更聪明”“那版好像更稳”的口水仗。

评测作为管理方法

评测远超发布前的打分仪式。它是组织资源分配的方法：帮助团队决定收集什么数据、优化哪个场景、保留哪个版本、放弃哪条路线，以及如何判断一次训练真的学到了目标能力。



图 18: 评测页把管理问题拆成连续追问：什么是合适的模型、别人表现怎样、我们表现怎样，最后落到“定义好的问题到合适解决方案”的闭环。²⁴

这个观点和第 4 章的“评测倒逼训练”相连，但本章强调的是管理含义。对研究团队，评测决定数据构造和训练方向；对产品团队，评测决定哪些能力值得放进主路径；对个人从业者，评测决定学习和判断的准星。不会评测的人，很容易被 demo、排行榜、社交媒体热词牵着走；会评测的人，至少能追问样本分布、成功标准、对照版本和失败案例。

Benchmark 也会制造盲点

讲者说训练模型本质上会围绕 benchmark 展开，这句话需要加上边界：benchmark 是强有力的优化靶子，也会把团队带向过拟合。好的评测体系要保留隐藏集、真实日志回放、人工复核、反例分析和跨场景权衡。只盯公开分数，最后可能得到一个会考试的模型，产品现场仍然不稳。

5.4 PMF 前埋伏：技术拐点来临前先站好位置

00:19:27–00:21:24 是本章最有行动感的一段。PMF，即 product-market fit，通常译作产品市场契合，指产品能力、用户需求、使用频率、付费意愿和传播路径之间形成了强匹配。讲者的说法是“PMF 到来前要埋伏”。这句话的重点不在投机，它强调技术供给和市场需求真正咬合之前，团队已经要完成位置选择和能力储备。

讲者先拿 Z.ai 举例：Z.ai 被确立为智谱的主战场，关键拐点被他说成 2025 年 7 月 28 日 GLM-4.5 发布。公开来源可以确认，Z.ai 于 2025 年 7 月 28 日发布 GLM-4.5，并把它定位为具备推理、代码和 agentic 能力的基础模型。²⁵ 至于演讲幻灯片里“与某个热门智能体打平”的具体对象，字

²⁴源视频时间区间：00:18:27–00:19:27。

²⁵Z.ai / PR Newswire, <https://www.prnewswire.com/news-releases/zai-releases-glm-4-5--setting-new-standards-for-a-i-performance-and-accessibility-302514803.html>; Z.ai Developer Docs, <https://docs.z.ai/release-notes/new-released>.

幕识别为“js Mark”，需要以图像 agent 对 00:19:32–00:20:00 幻灯片的视觉确认作为最终依据。当前文本只保留讲者的核心判断：流量到来时，产品必须已经有承接能力。

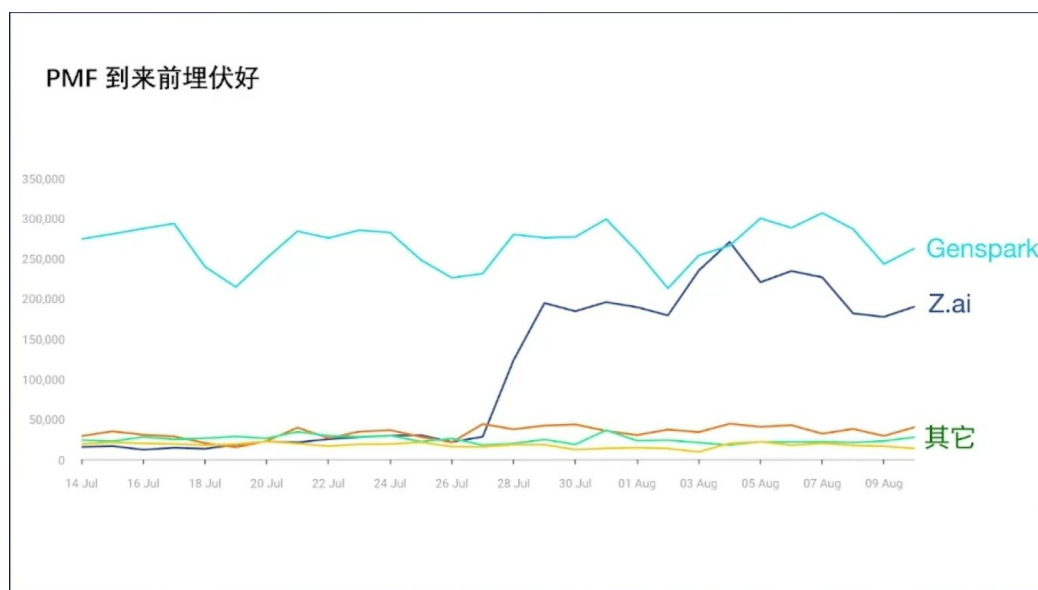


图 19: PMF 折线图展示了 Z.ai 在 GLM-4.5 发布后迅速抬升的拐点，讲者用它说明产品要在流量到来前具备承接能力；Cursor 与 NotebookLM 的例子可放在这张图后的文字中讨论。²⁶

“接得住”包含四层准备。第一，入口要清楚，用户来了知道从哪里开始。第二，核心场景要稳定，不能靠一次惊艳 demo 支撑长期使用。第三，评测和日志要闭环，团队能知道新用户为什么留下或流失。第四，供给侧能力要跟得上，包括模型成本、延迟、限流、错误恢复和客服机制。PMF 常常看起来像突然爆发，背后却是很多不显眼的预埋工作。

讲者又举了两个参考产品。第一个是 Cursor 与 Claude Sonnet 3.5。他的说法是 Cursor 押中了 Claude Sonnet 3.5，使 Tab 相关体验进入 PMF。公开资料能确认两件事：Anthropic 于 2024 年 6 月发布 Claude 3.5 Sonnet；Cursor 文档和社区资料显示 Cursor 支持 Claude 3.5 Sonnet，且 Cursor Tab 是其核心自动补全能力之一。²⁷ 但“哪一个模型、哪一个版本、哪一项 Tab 体验首次进入 PMF”属于讲者的产品判断，最终应结合幻灯片和当时产品数据理解。

第二个是 NotebookLM 与 Google 自家模型。字幕把模型名识别成“nether banana”，结合 2025 年 Google 官方博客，更可能指 Nano Banana，即 Google 的图像生成/编辑能力进入 Search、NotebookLM 等产品。²⁸ 这个例子说明平台型公司有一种特殊优势：模型团队和产品团队都在同一生态内，产品方可能更早知道自家模型能力将在哪个时间点变得可用，从而提前设计入口和场景。

PMF 前埋伏的反向概念

PMF 后追赶也常见：等市场已经证明需求，再快速复制功能、打渠道、拼价格。它的好处是确定性高，坏处是窗口期短、用户心智已被占据、技术债和体验细节很难补齐。PMF 前埋伏承担更高不确定性，换来的是当技术拐点出现时，产品、数据、评测和分发已经排好队。

²⁶源视频时间区间：00:19:31–00:20:36。

²⁷Anthropic, <https://www.anthropic.com/news/claude-3-5-sonnet>; Cursor Docs, <https://docs.cursor.com/models>; Cursor Docs, <https://docs.cursor.com/en/tab/overview>.

²⁸Google Blog, <https://blog.google/technology/ai/nano-banana-google-products/>; Google Labs, <https://blog.google/innovation-and-ai/models-and-research/google-labs/video-overviews-nano-banana/>.

²⁹源视频时间区间：00:20:39–00:21:24。



图 20: “PMF 到来前埋伏好”被拆成前瞻性洞察与技术储备：一边识别潜在需求和市场空白，一边提前构建模型适配层与可扩展产品架构。²⁹

对个人从业者，讲者给了两个准备方向。第一是前瞻性洞察：多看更本质的东西，例如论文、技术报告、长期布道者和连续失败的方向。这里的“本质”指底层约束是否在变化，例如模型成本下降、上下文和工具调用增强、视觉输入进入代码链路、评测从单题走向长程。第二是技术储备：多和模型厂、产品团队、行业内的人交流，尤其要理解失败案例。成功案例往往被包装得很顺，失败案例更能暴露边界条件、组织误判和真实瓶颈。

个人准备的核心判断

当一个风口已经肉眼可见，最好的位置通常已经被提前准备的人占住。个人能做的准备包括：持续读技术报告和论文，跟踪产品入口变化，建立自己的评测样本，理解失败案例，积累能在拐点到来时立刻投入的工程能力。

5.5 讲者结尾与行业速度

00:21:24–00:22:03 是收尾。常规联系方式可以压缩，但讲者的自我介绍和行业速度反思值得保留。他说自己在智谱负责模型发布、API、chat agent 等工作，同时也是 Coding Plan 的创始人。随后他补了一句很有信息量的话：智谱的 Coding Plan 现在已经不怎么卖了，这意味着大模型行业发展太快；半年前因为创立一个东西获得巨大成功，今天可能就要迈向下一个时代。

这段话给前面的 PMF 讨论加了现实温度。PMF 会随技术周期折旧。大模型行业里，模型能力、成本结构、交互方式和用户预期都在高速迁移。一个产品可能因为某次模型能力跃迁突然起飞，也可能因为下一次能力跃迁改变了用户入口而迅速失去优势。技术产品的寿命由“曾经解决过痛点”和“下一轮能力分布变化时能否重新定位”共同决定。

³⁰源视频时间区间：00:21:24–00:22:03。



图 21: 收尾页保留了讲者姓名、Z.ai 负责人身份和联系方式画面，可作为本章最后讨论行业速度与个人角色迁移的视觉锚点。³⁰

行业速度带来的残酷事实

在大模型应用里，半年前的成功经验很可能已经落后。旧产品形态、旧评测集、旧交互入口、旧成本假设，都可能在新型发布后失效。真正可迁移的资产是场景理解、评测能力、工程执行、用户信任和对技术拐点的敏感度。

从整场演讲看，讲者最后没有停在“GLM-5.1 很强”这类发布会结论上。他把模型训练、场景分类、评测体系和产品时机连成了一条链：先理解长程任务为什么出现，再解释模型如何训练，再说明评测怎样倒逼优化，最后要求从业者在 PMF 来临前准备好承接位置。这条链路也是本 PDF 最后应留下的实践判断。

5.6 本章小结

本章把长程智能体的技术主题落到产品化和个人行动。第一，研究员需要像产品经理一样理解用户、场景和边界，因为模型能力只有嵌入真实链路后才会显出价值和风险。第二，vision coding 的截图例子说明，用户行为常常会暴露输入通道缺口；截图保留了代码缩进、IDE 状态和错误位置，因而可能成为代码模型的重要上下文。第三，评测是 AI 时代的管理方法，它把体感判断拆成场景、权重、指标和版本选择，直接影响训练数据、产品路线和组织资源。

第四，PMF 前埋伏要求团队在市场显性爆发前就完成场景假设、技术储备、产品入口和评测闭环。Z.ai 与 GLM-4.5 的例子可以确认发布日期和模型定位，具体图表排名仍需以幻灯片为准；Cursor、Claude Sonnet 3.5、NotebookLM、Nano Banana 等例子也应写成讲者的产品判断，并用公开来源确认可确认的事实。第五，讲者最后关于 Coding Plan 的反思提醒读者：大模型行业的成功会迅速折旧，真正需要积累的是跨周期能力。

全章压缩

长程智能体同时考验模型能力、产品时机、场景选择和评测管理。场景意识决定 vision coding 这类输入通道是否进入训练；评测权重决定团队优化哪类用户任务；PMF 前埋伏决定 GLM-4.5 这类能力拐点出现时，产品能否接住流量。

6 总结与延伸

讲者的收尾给整场报告留下了一个清醒判断：大模型应用的成功会快速折旧。半年前因为 Coding Plan 这类产品获得的成绩，到今天可能已经进入下一轮迁移。这个判断把前五章连起来：长程智能体能力既是训练问题，也是评测问题、产品问题和组织判断问题。

6.1 核心定义的压缩

本 PDF 采用一个更精确的定义：长程智能体任务指完整智能体系统 A 围绕目标 G ，在行动 a_t 、观察 o_t 、状态 S_t 的循环中持续推进，并通过外部验证判断是否完成的任务。它的难点集中在四个方面：目标保持、状态连续、失败恢复、验收闭环。

一句话带走

Long Horizon 的关键在于过程可靠性。系统要在很多步之后仍然知道自己为什么出发、已经试过什么、哪些证据可靠、下一步怎样更接近可验证结果。

第 4 章引入 METR 的任务视界 $H_{p,D}(A)$ ，把“长”放到合格人类完成时间和成功概率上。这个量化方式能避免三个混淆：上下文长度只是信息容量，任务视界是任务难度尺度，自治视界是产品部署中离开人工干预后能推进多远。

6.2 从模型到系统

OpenAI 与 Anthropic 的公开资料共同提醒了一件事：长程能力落在完整 agent system 上。模型 M 很重要，但它必须和工具、指令、运行时、session、harness、sandbox、权限、日志和评测回路协同工作。一个只能生成漂亮局部结果的模型，在长任务里会遇到目标漂移、重复试错、状态膨胀和验收延迟；一个设计良好的智能体系统，会把失败记录成状态，把验证结果转成下一轮行动。

GLM-5.1 的训练叙事可以放在这个系统框架里理解。Mid-training 提供高质量代码、推理和长上下文/agent 数据；统一模型与自蒸馏减少专家模型之间的能力断裂；OPD 用跨阶段分布匹配缓解灾难性遗忘。这些技术动作的共同目标，是让模型在长链路中保持稳定偏好：会查证，会跑测试，会修正，会保留已验证事实。

6.3 从评测到产品

场景分类把长程能力从口号转成训练对象。用户日志告诉团队真实入口在哪里，权重重排告诉团队哪些任务更能代表能力，METR 式任务视界告诉团队任务尺度如何拉开。评测在这里像方向盘：它决定团队收集什么数据，优化什么场景，放弃什么虚高分数。

产品化部分给出的判断更直白。场景意识决定 vision coding 这类输入通道是否进入训练；评测权重决定团队优化哪类用户任务；PMF 前埋伏决定 GLM-4.5 这类能力拐点出现时，产品能否接住流量。长程智能体最终要交付的东西，通常是一段可被用户信任的过程，而非一次好看的答案。

拓展阅读：OpenAI: [Practices for Governing Agentic AI Systems](#), [A Practical Guide to Building Agents](#); Anthropic: [Building Effective Agents](#), [Managed Agents](#), [Measuring Agent Autonomy](#); 学术基准：METR / Kwa et al. 2025, SWE-bench, WebArena。